01. Introduction and Formal Languages

This section introduces the foundational concepts of compilation, formal languages, and automata theory, which form the theoretical basis for Lexical Analysis (Scanning).

---

1. Overview of Compilers

A **compiler** is a program that translates an executable program written in a source language into an equivalent program in a target language (often lower-level, such as assembly, machine code, or an intermediate representation like LLVM IR).

1.1 Compiler Architecture

Modern compilers are typically divided into three main stages (the "Three-Phase Design"):

1. **Front-End:** Understands the source code. Includes Lexical Analysis (Scanner), Syntax Analysis (Parser), and Contextual/Semantic Analysis (Type Checking). Produces an Intermediate Representation (IR).
2. **Optimizer (Middle-End):** Performs IR-to-IR transformations to improve performance, reduce code size, or optimize energy consumption. Involves Dataflow Analysis and Control Flow Analysis.
3. **Back-End:** Maps the IR to the target machine. Involves Instruction Selection, Register Allocation, and Instruction Scheduling.

---

2. Formal Languages Preliminaries

To build a compiler, we must formally define the programming language.

- **Alphabet (Σ):** A finite set of symbols (e.g., $\Sigma = \{0, 1\}$ or ASCII).
- **String:** A finite sequence of symbols from Σ. The empty string is denoted as ϵ.
- **Language (L):** A set of strings over a given alphabet. $L \subseteq \Sigma^*$, where Σ^* is the Kleene closure of Σ (all possible strings including ϵ).

2.1 Grammars

A generative grammar is defined as a 4-tuple $G = (N, \Sigma, P, S)$:

- N: Finite set of non-terminal symbols.
- Σ: Finite set of terminal symbols (disjoint from N).
- P: Finite set of production rules of the form $\alpha \to \beta$.
- $S \in N$: The start symbol.

2.2 The Chomsky Hierarchy

Languages are classified by the restrictiveness of their production rules:

1. **Type 0 (Recursively Enumerable):** Unrestricted grammar. Recognized by Turing Machines.
2. **Type 1 (Context-Sensitive):** Productions $\alpha A \beta \to \alpha \gamma \beta$. Recognized by Linear Bounded Automata.
3. **Type 2 (Context-Free):** Productions $A \to \gamma$. Recognized by Pushdown Automata (used for Parsing).

4. **Type 3 (Regular):** Productions $A \to aB$ or $A \to a$. Recognized by Finite Automata (used for Lexical Analysis).

---

3. Finite Automata

Finite automata are state machines used to recognize Regular Languages. They consume an input string symbol by symbol and change state.

3.1 Deterministic Finite Automata (DFA)

A DFA is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$:

- Q: Finite set of states.
- Σ: Input alphabet.
- $\delta : Q \times \Sigma \to Q$: The transition function. For every state and input symbol, there is exactly one next state.
- $q_0 \in Q$: The initial state.
- $F \subseteq Q$: The set of final (accepting) states.

A string w is accepted if the automaton, starting in q_0 and consuming w, halts in a state $q_f \in F$.

3.2 Non-Deterministic Finite Automata (NFA)

An NFA relaxes the determinism constraints. It is defined as $M = (Q, \Sigma, \delta, q_0, F)$, but the transition function maps to a *set* of states:

$$\delta : Q \times \Sigma \to \mathcal{P}(Q)$$

An NFA accepts a string if *at least one* valid computational path leads to an accepting state.

3.3 Automata with ϵ-Transitions (ϵ-NFA)

An ϵ-NFA allows transitions without consuming any input symbol.

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(Q)$$

ϵ-closure: The ϵ-closure of a state q, denoted ϵ-closure(q), is the set of all states reachable from q using only ϵ-transitions, including q itself.

---

4. Equivalences and Regular Expressions

4.1 Equivalence of NFA and DFA (Subset Construction)

Every NFA can be converted into an equivalent DFA that recognizes the same language. This is done via the **Subset Construction Algorithm** (also known as Powerset Construction).

Given an NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, the equivalent DFA $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ is constructed as:

- $Q_D = \mathcal{P}(Q_N)$ (Each state in the DFA represents a *set* of states in the NFA).
- $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$ for $S \in Q_D$ and $a \in \Sigma$.
- $F_D = \{S \in Q_D \mid S \cap F_N \neq \emptyset\}$.

(Implementation note: In lexer generators like `lex` or `flex` generating C/C++ code, computing the DFA explicit table during generation allows an extremely fast $O(|w|)$ scanning phase, avoiding the runtime overhead of tracking multiple active NFA states).

4.2 Regular Expressions (RE)

Regular expressions are an algebraic way to describe regular languages. The base operators are:

1. **Union ($A \cup B$ or $A + B$):** $L(A) \cup L(B)$
2. **Concatenation (AB):** $\{ab \mid a \in L(A), b \in L(B)\}$
3. **Kleene Star (A^*):** Zero or more occurrences of A. $\bigcup_{i=0}^{\infty} A^i$

Kleene's Theorem: A language is regular if and only if it can be described by a regular expression. Therefore, REs $\equiv$ ϵ-NFAs $\equiv$ NFAs $\equiv$ DFAs.

---

5. DFA Minimization

For any regular language, there exists a unique DFA with the minimum number of states.

5.1 Distinguishable States and Table-Filling Algorithm

Two states p and q are distinguishable if there exists a string w such that starting from p and reading w leads to an accepting state, while starting from q and reading w leads to a rejecting state (or vice-versa).

Minimization Algorithm:

1. Initialize a table marking all pairs (p, q) where $p \in F$ and $q \notin F$ (or vice-versa) as distinguishable.
2. Repeat: For any unmarked pair (p, q) and any symbol $a \in \Sigma$, if $(\delta(p, a), \delta(q, a))$ is marked, then mark (p, q).
3. Stop when no new pairs can be marked.
4. Unmarked pairs are **equivalent** and can be merged into a single state.

---

6. Proving Languages are NOT Regular: The Pumping Lemma

Not all languages are regular (e.g., $L = \{0^n 1^n \mid n \geq 0\}$ cannot be parsed by a Lexer because an FA has finite memory and cannot "count" arbitrarily high).

Pumping Lemma for Regular Languages:

If L is a regular language, there exists an integer $p > 0$ (the pumping length) such that every string $s \in L$ with length $|s| \geq p$ can be partitioned into three pieces, $s = xyz$, satisfying three conditions:

1. $|y| > 0$ (The pumped section is not empty).
2. $|xy| \leq p$ (The loop occurs within the first p characters).
3. For all $i \geq 0$, the string $xy^i z \in L$.

To prove a language is *not* regular, we use the Pumping Lemma in a proof by contradiction, demonstrating that an adversary can pick a string $s \in L$ where *no possible valid partition* satisfies all three rules simultaneously.

02. Context-Free Languages and Lexical Analysis

1. Lexical Analysis (Scanning)

The Lexical Analyzer (Scanner) is the first phase of the compiler front-end. Its primary job is to read the stream of characters making up the source program and group them into meaningful sequences called **lexemes**. For each lexeme, the scanner produces a **token** of the form: `⟨token-name, attribute-value⟩`

1.1 From Regular Expressions to Code

Lexical analyzers are often generated automatically by tools (like `lex` or `flex`). The generator takes a specification file containing Regular Expressions (RE) associated with semantic actions (usually C/C++ code).

The compilation process of a scanner is: 1. **RE to ϵ-NFA:** Thompson's Construction. 2. **ϵ-NFA to DFA:** Subset Construction Algorithm. 3. **DFA to Minimized DFA:** Hopcroft's Algorithm (Table-filling). 4. **DFA to Code:** The minimized DFA is encoded as a transition table (a 2D array mapping `State x Character -> Next State`) driven by a simple `while` loop, yielding an $O(|w|)$ scanning phase.

---

2. Context-Free Grammars (CFG)

Regular languages cannot express constructs with unbounded nesting (like balanced parentheses or matching `if-else` blocks). To define the syntax of a programming language, we use Context-Free Grammars (Type 2 in the Chomsky Hierarchy).

> **Definition (CFG):** A CFG is a 4-tuple $G = (\Sigma, N, S, P)$, where: * Σ: Finite set of terminal symbols (the alphabet/tokens). * N: Finite set of non-terminal symbols (syntactic variables). $\Sigma \cap N = \emptyset$. * $S \in N$: The start symbol. * P: Finite set of productions of the form $A \to \alpha$, where $A \in N$ and $\alpha \in (\Sigma \cup N)^*$.

2.1 Derivations and Parse Trees

A grammar derives strings by repeatedly replacing non-terminals using the production rules. * $\Rightarrow$ **(Derives in one step):** If $A \to \gamma \in P$, then $\alpha A \beta \Rightarrow \alpha\gamma\beta$. * $\Rightarrow^*$ **(Derives in zero or more steps):** The reflexive and transitive closure of $\Rightarrow$.

The language generated by G is $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$.

Parse Tree: A graphical representation of a derivation. The root is S, internal nodes are non-terminals, and leaves are terminals (or ϵ).

2.2 Leftmost and Rightmost Derivations

- **Leftmost Derivation:** At each step, the leftmost non-terminal is replaced.
- **Rightmost Derivation:** At each step, the rightmost non-terminal is replaced.

2.3 Ambiguity

A grammar is **ambiguous** if it can generate the same string with more than one distinct parse tree (or equivalently, more than one leftmost/rightmost derivation). *Ambiguity is fatal for compilers because it implies multiple valid semantic interpretations for the same code.* To resolve ambiguity, we typically rewrite the grammar to enforce precedence and associativity rules (e.g., separating expressions into `Expr`, `Term`, and `Factor`).

---

3. Pushdown Automata (PDA)

A Pushdown Automaton is the computational model equivalent to a CFG. It is essentially an ϵ-NFA augmented with a Stack memory.

3.1 Formal Definition

A PDA is defined as $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$: * Q: Finite set of states. * Σ: Input alphabet. * Γ: Stack alphabet. * $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \to \mathcal{P}(Q \times \Gamma^*)$: Transition function. * q_0: Initial state. * $Z_0 \in \Gamma$: Initial stack symbol. * $F \subseteq Q$: Set of final states.

3.2 Acceptance Criteria

A PDA can accept a string in two equivalent ways: 1. **Acceptance by Final State:** The automaton consumes all input and ends in a state $q_f \in F$. 2. **Acceptance by Empty Stack:** The automaton consumes all input and leaves the stack empty (useful for proving equivalence with CFGs).

---

4. Properties of Context-Free Languages

4.1 Closure Properties

Context-Free Languages are **closed** under: * Union ($L_1 \cup L_2$) * Concatenation ($L_1 L_2$) * Kleene Star (L^*)

CFLs are **NOT closed** under: * Intersection ($L_1 \cap L_2$). *However, the intersection of a CFL and a Regular Language is always a CFL.* * Complement ($\overline{L}$).

4.2 The Pumping Lemma for CFGs

Just as finite automata cannot count arbitrarily, PDAs have limitations (they can only compare two quantities using the stack, not three). The Pumping Lemma for CFGs is used to prove that languages like $L = \{a^n b^n c^n \mid n \geq 0\}$ are **not** Context-Free.

Theorem: If L is a Context-Free Language, there exists an integer $k > 0$ such that every string $z \in L$ with $|z| \geq k$ can be partitioned into five substrings $z = uvwxy$ satisfying: 1. $|vwx| \leq k$ (The pumped section is bounded). 2. $|vx| > 0$ (At least one of v or x is non-empty). 3. For all $i \geq 0$, the string $uv^iwx^iy \in L$.

If an adversary can pick a string $z \in L$ where *no possible valid partition* allows uv^iwx^iy to stay in L for all i, then the language is not Context-Free (and thus requires a Context-Sensitive Grammar or a Turing Machine).

03. Parsing Techniques

This section covers the second fundamental phase of the compiler front-end: Syntax Analysis (Parsing). The parser's job is to take the sequence of tokens produced by the scanner and verify if it can be generated by the Context-Free Grammar (CFG) of the language, simultaneously building a syntax tree (Parse Tree).

---

1. Introduction to Parsing

There are two primary approaches to building a parse tree: 1. **Top-Down Parsing:** Builds the tree from the root (start symbol) down to the leaves (the tokens). It attempts to apply productions by expanding the leftmost non-terminal (Leftmost derivation). 2. **Bottom-Up Parsing:** Builds the tree from the leaves up to the root. It attempts to recognize the right-hand sides of productions and reduce them to the left-hand side (Reverse Rightmost derivation).

---

2. Top-Down Parsing and FIRST/FOLLOW Sets

Predictive Top-Down parsers (like LL(k)) make decisions on which production to apply by looking at the next k tokens in the input (lookahead). To automate this choice, two fundamental functions are defined on the grammar symbols:

2.1 The FIRST Set

`FIRST(\alpha)` is the set of terminals that begin the strings derivable from `\alpha`. If `\alpha` can derive the empty string, then ϵ belongs to `FIRST(\alpha)`.

2.2 The FOLLOW Set

`FOLLOW(A)` is the set of terminals that can appear immediately to the right of the non-terminal `A` in any sentential form derived from the start symbol. * The end-of-file symbol (e.g., `EOF` or `$`) belongs to `FOLLOW(S)`. * It is used to decide what to do when a production derives ϵ or when the end of a right-hand side rule is reached. If `FIRST(A)` contains ϵ, the parser considers the symbols in `FOLLOW(A)` to expand `A`.

---

3. Bottom-Up Parsing (Shift-Reduce)

Bottom-Up parsing (typically LR) is more powerful than Top-Down and can handle a much larger class of grammars. The algorithm maintains a stack and interacts with the input through two main operations:

1. **Shift:** Takes the current token from the input and pushes it onto the stack.
2. **Reduce:** When the symbols at the top of the stack match the right-hand side (rhs) of a production, they are popped from the stack and replaced with the non-terminal found on the left-hand side (lhs) of the production.

---

4. Construction of LR(1) Tables

An LR(1) parser uses a state and a lookahead symbol (1 token) to make optimal parsing decisions. The state of the parser is represented by a set of **Items**. Limited right context is enough to pick the correct actions.

4.1 LR(1) Items

An LR(1) item for a production $A \to \beta$ with lookahead a has the form:

$$[A \to B_1 B_2 \bullet B_3, a]$$

The dot ($\bullet$) indicates how much of the production has already been recognized. The lookahead symbol serves a critical bookkeeping function to determine reductions: * In an item with the dot in the middle $[A \to \beta \bullet \gamma, a]$, the lookahead has no direct use for the reduction. * In an item with the dot at the end $[A \to \beta \bullet, a]$, a lookahead equal to a implies that the parser must perform a reduction using the production $A \to \beta$.

4.2 The Canonical Collection Algorithm

To generate the LR(1) parsing table, it is necessary to build the entire canonical collection of sets of LR(1) items (which represent the states of the pushdown automaton).

1. **Initialization:** Start with an augmented grammar by adding a new root production $S' \to S$. Begin with the initial state s_0, computing the closure of the first item:

   $$s_0 \leftarrow \text{closure}([S' \to \bullet S, \text{EOF}])$$

   This step derives all equivalent items by expanding the non-terminals immediately to the right of the dot.

2. **Computing Goto and Reaching a Fixed Point:** While the collection of states S is still changing, apply the following process:
   - For each state $s_j \in S$ and for each symbol X (both terminal and non-terminal), compute the transition set $t \leftarrow \text{goto}(s_j, X)$.
   - If the resulting set t is not already in the collection S, compute its closure and add it as a new state s_k.
   - Record the transition from the source to the destination state: $s_j \xrightarrow{X} s_k$.

04. Intermediate Representation & Code Shape

This section marks the transition from the front-end to the middle-end of the compiler. We are on the cusp of the art, science, and engineering of compilation. While scanning and parsing are direct applications of automata theory, and context-sensitive analysis is mostly software engineering, the design of the Intermediate Representation (IR) dictates the efficiency of the optimization and code generation phases.

---

1. Intermediate Representations (IR)

An IR is an internal structure used by the compiler to represent the source program during translation. A well-designed IR must be expressive enough to capture the source program's semantics but low-level enough to map cleanly to the target architecture.

1.1 Structural / Graphical IRs

These IRs represent the code as graphs or trees, ideal for capturing the hierarchical nature of source code. * **Abstract Syntax Trees (AST):** A tree where internal nodes are operators and leaves are operands. It abstracts away syntactic details (like parentheses). * **Directed Acyclic Graphs (DAG):** Similar to an AST, but identical subtrees are shared. This explicitly encodes Common Subexpression Elimination (CSE) at the expression level. * **Control Flow Graphs (CFG):** A directed graph where each node is a **Basic Block** (a maximal sequence of straight-line code with one entry point and one exit point). Edges represent control flow (branches, loops).

1.2 Linear IRs

Linear IRs consist of a sequence of instructions executed sequentially, closely resembling low-level machine code (e.g., LLVM IR or ILOC). * **Three-Address Code (3AC):** Instructions take the form `x = y op z`. Complex expressions are broken down using temporary variables. * **Static Single Assignment (SSA):** A strict property applied to linear IRs where every variable is defined exactly once. When control flow paths merge, a ϕ-function (phi-function) is used to multiplex the values (e.g., `x_3 = phi(x_1, x_2)`). This greatly simplifies data-flow analysis.

---

2. The Procedure Abstraction

Before generating code, the compiler must establish a standard runtime environment to manage procedure calls, memory allocation, and variable scoping.

2.1 Activation Records (Stack Frames)

Every time a function is called, a new Activation Record is pushed onto the call stack. It typically contains: 1. **Actual Parameters:** Arguments passed by the caller. 2. **Return Address:** Where to resume execution in the caller. 3. **Control Link (Dynamic Link):** A pointer to the caller's activation record (used to restore the stack pointer). 4. **Access Link (Static Link):** A pointer to the lexically enclosing environment,

necessary to resolve non-local variables in languages with nested scopes. 5. **Saved Registers:** Registers whose values must be preserved across the call. 6. **Local Variables:** Storage for variables declared within the function.

2.2 Calling Sequences

The protocol for invoking a procedure is split between the caller and the callee. * **Prologue:** Code executed at the beginning of the callee to set up its frame pointer, allocate local variables, and save necessary registers. * **Epilogue:** Code executed at the end of the callee to restore saved registers, place the return value in a standard location, and restore the stack pointer before executing the return jump.

---

3. Code Shape

"Code Shape" refers to the specific sequences of IR instructions chosen to implement high-level language constructs. Lowering an AST to an IR requires making concrete choices about memory layout and execution order.

3.1 Expressions

Expressions are evaluated by walking the AST (usually in post-order) and generating temporary variables for intermediate results. *AST for `a + (b * c)` becomes:*

```
// lowercase comments kept in english for code clarity
t1 = b * c
t2 = a + t1
```

Compilers must carefully manage the types and sizes of operands, particularly when dealing with pointers or low-level type casting (e.g., standard C++ pointer arithmetic implicitly multiplies the offset by `sizeof(type)`).

3.2 Boolean Expressions and Control Flow

Booleans can be evaluated in two ways:

1. **Numerical Representation:** `true` is 1, `false` is 0. Operators compute integer values.
2. **Short-Circuit Evaluation (Control Flow):** The boolean result is implicit in the control flow. Instead of computing a value, the code jumps to a "true" or "false" branch as soon as the outcome is known.

Code shape for an `if-then-else` statement:

```
  // evaluate condition
  t1 = <condition>
  cbr t1 -> L_then, L_else  // conditional branch
L_then:
  // evaluate then block
  br L_end                  // unconditional branch
L_else:
  // evaluate else block
L_end:
```

3.3 Arrays

Array accesses require generating address calculations.

For a 1-dimensional array `A[i]`, the memory address is:

```
Address = base_address(A) + (i * element_size)
```

For a 2-dimensional array `B[i][j]` (Row-Major order, like in C/C++):

```
Address = base_address(B) + ((i * num_columns) + j) * element_size
```

3.4 Case / Switch Statements

Compilers employ different strategies for `switch` blocks depending on the density of the case values:

- **Sequential `if-else` (Linear Search):** Best for a small number of cases.
- **Binary Search:** Best if the cases are sparse but numerous.
- **Jump Tables:** If the case values are dense (e.g., 1, 2, 3, 4), the compiler stores code labels in an array and performs an indirect jump in $O(1)$ time: `goto JumpTable[value]`.

05. Data-Flow Analysis & Fixpoint Theory

This section explores the mathematical frameworks and algorithms used in the middle-end of a compiler to gather information about the program's behavior. This information is crucial for performing safe optimizations.

---

1. Approaches to Program Analysis

Program analysis can be performed using various techniques, each differing in focus and semantic foundations. A family of techniques includes: * **Data Flow Analysis:** Based on the Control Flow Graph (CFG). * **Constraint Based Analysis:** Extracts inclusions (constraints) from the program flow. * **Type and Effect Systems:** Uses annotated base types and type constructors. * **Abstract Interpretation:** A generalized mathematical framework for approximating program semantics.

1.1 The Data-Flow Approach

Data-flow analysis constructs an equation system in terms of a specific property at the entry and exit of each node in the CFG. The solution to this system of equations is found by computing a fixpoint (either a least fixpoint or a greatest fixpoint, depending on the approximation direction).

---

2. Theoretical Foundations: Fixpoint Theory

To guarantee that our equation systems converge to a solution, we rely on domain theory and partial orders.

2.1 Partial Orders and Bounds

A domain D requires a way to compare elements, typically denoted by a partial order relation $\sqsubseteq$. If $x \sqsubseteq y$, it means x is a less precise approximation of y, and y is more accurate than x.

Given a subset $Q \subseteq S$ in a partially ordered set: * An element is an **upper bound** if it is greater than or equal to all elements in Q. * The **Least Upper Bound (lub)**, if it exists, is the smallest of all upper bounds. If Q is a finite subset of $\mathbb{N}$ under $\leq$, lub $Q = \max Q$.

2.2 Complete Partial Orders (CPO)

A Complete Partial Order (CPO) ensures that iterative approximation processes will converge. We iteratively build a sequence of approximations:

$$x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \cdots \sqsubseteq x_n \sqsubseteq \ldots$$

where each step represents a refined analysis state. The process halts when a fixpoint is reached (i.e., $x_k = x_{k+1}$).

---

3. Liveness Analysis (Live Variables)

Liveness analysis determines which variables hold values that may be read later in the execution. This is a **backward analysis**.

3.1 Definitions

- **Definition (`def[n]`):** An assignment to a variable (L-value usage) in node n.
- **Use (`use[n]`):** A usage of a variable's value (R-value usage) in a command.
- **Liveness Property:** A variable x is live along an arc $e \to f$ if there exists a real execution path from e to some node n such that $x \in \text{use}[n]$, and for any intermediate node n', $x \notin \text{def}[n']$.

3.2 Data-Flow Equations

The sets are computed iteratively until they stabilize: * `in[n] = use[n] U (out[n] - def[n])` * `out[n] = U { in[m] | m \in post[n] }`

3.3 Conservative Approximation

In static analysis, "conservative" means erring on the side of safety. In liveness analysis, if $x \in \text{in}[n]$, the variable *could* be live. If $x \notin \text{in}[n]$, it is *definitely dead*. The analysis may erroneously derive that a variable is live, but it is not allowed to erroneously derive that a variable is dead.

$$\text{in}[n] \supseteq \text{live-in}[n] \quad \text{and} \quad \text{out}[n] \supseteq \text{live-out}[n]$$

---

4. Reaching Definitions

This analysis determines which variable definitions (assignments) can reach a given point in the program without being overwritten. This is a **forward analysis**.

4.1 Kill and Gen Sets

- `killRD[p]`: The set of all other definitions of a variable x in the program, activated if node p defines x.
- `genRD[p]`: The specific definition of x generated at node p.

4.2 Data-Flow Equations

- For the initial node: `RDentry(p) = {(x, ?) | x \in VARS}`.
- For other nodes: `RDentry(p) = U { RDexit(q) | q \in pre[p] }`.
- Exit state: `RDexit(p) = (RDentry(p) \ killRD[p]) U genRD[p]`.

---

5. Available Expressions

Available expressions analysis determines which complex expressions (like `a+b`) have already been computed and not subsequently invalidated by reassignments to their operands. It is used for Common Subexpression Elimination.

5.1 Kill and Gen Sets

- $killAE[p]$: An expression e is killed at point p if a variable occurring in e is modified by the command at p.
- $genAE[p]$: An expression e is generated at p if it is evaluated in p and no variable in e is modified within p.

5.2 Data-Flow Equations

Unlike Liveness and Reaching Definitions (which use the Union operator $\cup$), Available Expressions uses the Intersection operator $\cap$ because an expression is available only if it is available along *all* incoming paths. * $AEentry(p) = \emptyset$ (if p is initial). * $AEentry(p) = \cap AEexit(q) | q \in pre[p]$ (otherwise). * $AEexit(p) = (AEentry(p)\ killAE(p)) U genAE(p)$.

06. Abstract Interpretation & Advanced Static Analysis

This section explores Abstract Interpretation, a generalized mathematical framework for static analysis, and other advanced techniques to approximate program semantics and prove correctness.

---

1. Advanced Approaches to Static Analysis

Beyond Data-Flow Analysis, the compiler middle-end can employ other analytical frameworks to gather information about the program:

1.1 Constraint-Based Analysis

Instead of solving equations iteratively over a CFG, this approach extracts a number of inclusions (equations or constraints) out of the program. * It considers the property at the entry and exit of a node and encodes the information of the flow of the CFG using constraints. * The analysis is automated by first extracting constraints from the program and then computing the least solution to those constraints.

1.2 Type and Effect Systems

This technique relies on annotated base types and annotated type constructors. * Statements are assigned types that represent the type of states. * A specification (annotated type system with axioms and rules) is implemented by extracting constraints and computing their least solution.

---

2. Abstract Interpretation Framework

Abstract Interpretation is based on interpreting the program's semantics with abstract values instead of concrete ones. It is used to discern program behavior and prove correctness without executing the code.

2.1 Concrete vs. Abstract Domains

- **Concrete Domain:** The actual mathematical space of the program's values. For instance, the concrete domain of sets of integers is defined by the partial order $\langle \mathcal{P}(\mathbb{Z}), \subseteq \rangle$.
- **Abstract Domain:** A simplified mathematical space. For example, a domain capturing only the sign of integers: $A = \{\top, \{+\}, \{-\}, \bot\}$.

2.2 Abstraction and Concretization Functions

To map values between domains, we define a Galois Connection using two functions: 1. **Abstraction (α):** Maps a concrete set to an abstract value. $\alpha(S) = \cup\{sign(s) \mid s \in S\}$. For example, $\alpha(\{2, 7\}) = \{+\}$ and $\alpha(\{-1, 3\}) = \top$ (where $\top$ means "unknown" or "any sign"). 2. **Concretization (γ):** Maps an abstract value back to a concrete set. $\gamma(X) = \{x \in \mathbb{Z} \mid sign(x) \in X\}$. For example, $\gamma(\top) = \mathbb{Z}$ and $\gamma(\bot) = \emptyset$.

2.3 Trace-Based Operational Semantics

A program's operational semantics can be written as a trace updating a program-point and storage-cell pair. The abstract transition rules are synthesized from the original ones: $p_i, a \longrightarrow p_j, \alpha(v')$ if $v \in \gamma(a)$ and $p_i, v \longrightarrow p_j, v'$. This recipe ensures that every transition in the concrete semantics is safely simulated by one in the abstract semantics.

---

3. Abstract Operators and Loss of Information

Operations on concrete values must be translated into abstract operations.

3.1 Defining Abstract Operations

Consider multiplication over $\mathcal{P}(\mathbb{Z})$: $\{3, 5\} * \{-1, -3\} = \{-3, -9, -5, -15\}$. In the abstract sign domain, the operator $\times$ is defined as: * $\{+\} \times \{+\} = \{+\}$ * $\{+\} \times \{-\} = \{-\}$ * $\{-\} \times \top = \top$

3.2 The Loss of Precision

Because abstract domains approximate sets of values, performing abstract operations can lead to a loss of information. * **Concrete evaluation:** $(\{2\} + \{2\}) - \{3\} = \{1\}$ (which corresponds to $\{+\}$). * **Abstract evaluation:** $(\{+\} + \{+\}) - \{+\} = \{+\} - \{+\} = \top$. The result $\top$ is sound (it contains $\{1\}$), but imprecise. Due to this loss of precision, the analysis might fail to decide termination for specific inputs.

---

4. Correctness, False Alarms, and Domains

The goal of static analysis is to prove that the reachable states of a program (blue area) do not intersect with the error states (red area), meaning blue $\cap$ red $= \emptyset$.

4.1 False Positives vs. False Negatives

- **Unsound Analysis:** If an analysis incorrectly claims a state is unreachable when it is actually reachable, it yields a **False Negative**. The program is bugged, but the analysis misses it.
- **Conservative Analysis:** Static analysis must be sound. It over-approximates the reachable states. If the over-approximation intersects with the error states while the concrete states do not, it triggers a **False Positive (False Alarm)**.

4.2 Choosing the Right Domain

The choice of abstract domain dictates the precision: * **Interval Domain:** Approximates variables independently (e.g., $x \in [0, 10]$). It captures rectangular bounding boxes. This can lead to false alarms if the safe region requires relational tracking (green $\cap$ red $\neq \emptyset$). * **Polyhedra Domain:** Tracks linear relations between variables (e.g., $x + y \leq 4$). It captures polygonal shapes and can prove correctness where the interval domain fails (cyan $\cap$ red $= \emptyset$).

---

5. Widening (∇)

When using domains like Intervals or Polyhedra, loops can generate infinite ascending chains of approximations (e.g., an index i growing at each iteration: $[0,0], [0,1], [0,2] \ldots$). To guarantee that the fixpoint computation terminates, Abstract Interpretation introduces the **Widening** operator. When the analysis detects that bounds keep growing across loop iterations, the widening operator forces convergence by extrapolating the bound to infinity (e.g., jumping to $\top$ or $[0, +\infty]$).

07. Register Allocation

Register allocation is one of the most critical phases in a compiler's back-end. The objective is to assign a potentially unbounded number of virtual registers (used in the intermediate representation) to a finite set of physical registers provided by the target processor architecture. The efficiency of this phase heavily dictates the performance of the compiled code, especially in low-level languages.

---

1. Static Single Assignment (SSA) Form

To simplify allocation and data-flow analysis, the intermediate representation is typically converted into SSA form. In this form, every variable is defined (assigned) exactly once in the program text.

When the control flow diverges and then merges (e.g., after an `if-else` block), the compiler must resolve the ambiguity of which variable definition is valid. To achieve this, it inserts **ϕ-functions** (phi-functions). * At each ϕ-function, the compiler takes the union of the arguments coming from the converging control flow edges.

2. Constructing Live Ranges

To proceed with the actual allocation, the compiler calculates how long each variable "lives" within the program (the period between its definition and its last use).

- Variables must be renamed to accurately reflect these new "live ranges".
- Arguments belonging to the same ϕ-function must be united together to form a cohesive, continuous live range.
- An example of identifying and grouping live ranges after these unions can be expressed as: `{LRa=a0, LRb=b0, LRc=c0, LRd=d0 d1 d2}`.

3. Global Allocation and Graph Coloring

Global register allocation (which operates over the entire Control Flow Graph of a procedure) is typically modeled as a **Graph Coloring** problem, which is known to be NP-complete.

1. **Interference Graph:** An undirected graph is constructed where each node represents a Live Range. An edge exists between two nodes if their Live Ranges "interfere"—meaning they are active (live) simultaneously at some point in the program. Two connected nodes cannot share the same physical register.
2. **Coalescing:** If two Live Ranges do not interfere and are connected by a copy instruction (e.g., `x = y`), their nodes can be merged to safely eliminate the redundant copy instruction.
3. **Spilling:** If the graph requires more "colors" (physical registers) than are available on the architecture, some Live Ranges must be "spilled" (saved to memory, typically on the stack). The compiler inserts load and store instructions, fragmenting the Live Range into smaller, shorter-lived portions, and repeats the coloring process.
4. **Coloring:** Registers (colors) are assigned to nodes such that no two adjacent nodes share the same color.

```cpp
#include <vector>
#include <set>

struct live_range {
    int id;
    std::set<int> interferences;
    int assigned_register = -1;
};

// add interference edge between two live ranges
void add_interference(live_range& a, live_range& b) {
    a.interferences.insert(b.id);
    b.interferences.insert(a.id);
}

// compute coloring for available physical registers
void color_graph(std::vector<live_range>& ranges, int num_physical_regs) {
    // k-coloring algorithm logic
}
```

08. Laboratory (MiniImp & MiniFun)

This section outlines the laboratory project, which involves implementing the semantics, analysis, and optimization of two simple programming languages: MiniImp (imperative) and MiniFun (functional).

---

1. Project Overview

The project is divided into two main parts: 1. A minimal imperative language (MiniImp) with analysis and optimization. 2. A minimal functional language (MiniFun) with a type system.

The final submission requires two artifacts: clean, commented code and a report. The report must contain a guide on how to run the code and detailed motivations behind implementation choices and problem resolutions.

---

2. MiniImp: Imperative Language

MiniImp operates by reading and updating a memory σ.

2.1 Syntax

A program is defined by the following grammar: * `prog ::= def main with input <var> output <var> as <cmd>` * `cmd ::= skip | <var> := <expr> | <cmd>; <cmd> | if <bexp> then <cmd> else <cmd> | while <bexp> do <cmd>` * `expr ::= <var> | <num> | <expr> + <expr> | <expr> - <expr> | <expr> * <expr>` * `bexp ::= true | false | <bexp> and <bexp> | not <bexp> | <expr> < <expr>`

2.2 Semantics and Memory

Abstractly, a memory associates locations (variables) to values (integers). Because some variables can be undefined, a memory is modeled as a partial function $\sigma : X \rightharpoonup Z$. The semantics are given by deduction rules (reductions): * Arithmetical expressions: $\langle \sigma, e \rangle \to_e n$ * Boolean expressions: $\langle \sigma, b \rangle \to_b n$ * Commands: $\langle \sigma, c \rangle \to_c \sigma'$ * Programs: $\langle p, n \rangle \to_p n'$

For instance, the sequential execution of commands updates the memory step by step:

$$\frac{\langle \sigma, c_1 \rangle \to_c \sigma_1 \quad \langle \sigma_1, c_2 \rangle \to_c \sigma_2}{\langle \sigma, c_1; c_2 \rangle \to_c \sigma_2}$$

2.3 Pitfalls: Deadlocks and Non-termination

- **Deadlocks:** A program may fail and reach an erroneous state where the semantics is undefined. In MiniImp, a deadlock occurs specifically when a variable is undefined during evaluation.
- **Non-termination:** Constructs like the `while` loop can cause non-termination (e.g., executing `while true do x := 1` yields an infinite derivation tree).

---

3. MiniFun: Functional Language

MiniFun introduces functional programming paradigms, requiring the management of environments, closures, and type inference.

3.1 Closures and Environments

When evaluating a function, the semantics must capture the environment at the time of evaluation. * For a standard abstraction `fun x => t`, the resulting closure is `(x, t, \rho)`. * For recursive functions defined via `letfun f x = t`, the closure also stores the function's own name: `(f, x, t, \rho)`.

3.2 Static Analysis and Type System

Static analysis aims to prove properties about the program's behavior without executing it, avoiding run-time errors such as undefined variables or applying addition between booleans and integers.

Type checking involves propagating constraining information across the Abstract Syntax Tree. The type-checking implementation requires defining data structures for monotypes, polytypes, and substitutions, as well as implementing `inst`, `gener`, and `unify` functions.

Algorithm W Step-by-Step for `if-then-else` (ITE): 1. Check the first argument. 2. Update the context with the substitution and check the second argument. 3. Update the context with the substitution and check the third argument. 4. Unify the first argument with `bool`. 5. Unify the second and third arguments.

```
// type structures and environment map
struct type { /* ... */ };
using substitution = std::map<std::string, type>;

// algorithm w unification step
substitution unify(const type& t1, const type& t2) {
    // perform standard unification
    return {};
}

<div style="page-break-after: always;"></div>

# 09. Advanced Bottom-Up Parsing

This section expands on the concepts of Shift-Reduce parsing, providing a rigorous definition of LR(1) :

----
```

1. Bottom-Up Parsing Overview

Bottom-up parsers, such as LR(1), build the syntax tree from the leaves toward the root. They handle a r
As the input is consumed from left to right, the parser encodes all possible derivations in an internal

1.1 The Shift-Reduce Mechanism
The parser state is maintained using a stack and the input buffer.
* **Shift:** Moves the next input token onto the stack.
* **Reduce:** When the top symbols on the stack match a production's RHS, they are popped and replaced b
* **Accept:** The input is fully consumed, and the stack contains only the start symbol.

---

2. LR(1) Items and State Representation

To track the parsing progress, we use the concept of an **LR(1) item**.

> **Definition (LR(1) Item):**
> An LR(1) item is a pair $[P, a]$, where P is a production $A \to \alpha \bullet \beta$, and a is a

The dot ($\bullet$) indicates the portion of the RHS that has already been matched and pushed onto the s
* $[A \to \bullet \beta, a]$: No symbols of this production have been matched yet.
* $[A \to \alpha \bullet \beta, a]$: The string derived from α is currently on top of the stack.
* $[A \to \gamma \bullet, a]$: The entire RHS has been matched. If the next input token is exactly a,

---

3. Constructing the Canonical Collection

To build the parsing tables, we must compute the states of the parser, which are sets of LR(1) items. Tw

3.1 The `closure(s)` Function
Given a set of items s, `closure(s)` adds all items that represent potential valid derivations expandi

```text
// compute closure of a set of lr(1) items
function compute_closure(initial_set):
    closure_set = initial_set
    changed = true

    while changed == true:
        changed = false

        // find new items to add
        for each item [A -> alpha . B beta, a] in closure_set:
            for each production B -> gamma in grammar:

                // compute lookahead
                for each terminal b in FIRST(beta + a):
                    new_item = [B -> . gamma, b]
                    if new_item not in closure_set:
                        add new_item to closure_set
                        changed = true
```

```
    return closure_set
```

3.2 The `goto(s, X)` Function

The `goto` function computes the transitions between states. Given a state s (a set of items) and a grammar symbol X (terminal or non-terminal):

`goto(s, X) = closure({ [A -> \alpha X \bullet \beta, a] | [A -> \alpha \bullet X \beta, a] \in s }).`

It represents the new state reached after recognizing X.

---

4. ACTION and GOTO Tables

Once the canonical collection of sets of LR(1) items (states $s_0, s_1, \ldots, s_n$) is built, we construct two tables:

4.1 ACTION Table

The ACTION table dictates the operation for a state given a lookahead token.

- **Shift s_k:** If $[A \to \alpha \bullet a\beta, b] \in s_i$ and `goto(`s_i, a`)` $= s_k$, set `ACTION[`s_i, a`] = shift` s_k.
- **Reduce $A \to \gamma$:** If $[A \to \gamma\bullet, a] \in s_i$ (and $A \neq S'$), set `ACTION[`s_i, a`] = reduce` $A \to \gamma$.
- **Accept:** If $[S' \to S\bullet, \text{EOF}] \in s_i$, set `ACTION[`$s_i, \text{EOF}$`] = accept`.

4.2 GOTO Table

The GOTO table determines the next state after a reduction.

- If `goto(`s_i, A`)` $= s_j$ for a non-terminal A, then `GOTO[`s_i, A`] =` s_j.

10. Context Sensitive Analysis

This section explores the transition from syntax analysis to semantic verification. While parsers can verify the structure of a program, they are fundamentally limited when dealing with context-dependent properties.

---

1. The Limits of Context-Free Grammars

Context-Free Grammars (CFGs) are excellent for defining the syntax of a language but lack the expressive power to enforce contextual and semantic rules. Properties that are not purely syntactic and depend on the surrounding context include: * **Variable Declarations:** Checking if a variable `x` has been declared before its use. * **Type Checking:** Ensuring type consistency in expressions (e.g., preventing addition between a boolean and an integer). * **Scope Rules:** Resolving variable shadowing and scoping across different blocks. * **Array Bounds:** Verifying if array access indices are legal. * **Function Signatures:** Checking if a function is called with the correct number and types of arguments.

Answering these questions requires non-local information (e.g., looking up a symbol table) and computation during the parsing phase.

---

2. Attribute Grammars

To solve the limitations of CFGs, we introduce **Attribute Grammars**. This formal method extends standard grammars by attaching attributes (values or properties) to the syntactic categories (non-terminals) and defining rules to compute these attributes alongside the productions.

2.1 Types of Attributes

Attributes govern how information flows through the parse tree:

1. **Synthesized Attributes:** Information flows *bottom-up*. The attribute of a parent node is computed from the attributes of its children. They are highly compatible with bottom-up parsers (like LR) because they can be evaluated at the exact moment a reduction occurs.
2. **Inherited Attributes:** Information flows *top-down* or sideways. The attribute of a child node is computed from the attributes of its parent or siblings.

2.2 Example: Binary Number Evaluation

Consider a grammar generating binary strings (e.g., `-1010` or `+10`). We want to compute the decimal value. * **`position` (Inherited):** Flows down from the root to the leaves. It tells a bit what its positional weight is ($2^0, 2^1, \ldots$). * **`value` (Synthesized):** Flows up from the leaves to the root. A leaf computes its value (`bit * 2^position`) and passes it up, where the parent sums the values of its children.

```
// ast node demonstrating attribute flow
struct expr_node {
    int inherited_pos = 0;
    int synthesized_val = 0;

    virtual void compute_inherited(int parent_pos) = 0;
    virtual void compute_synthesized() = 0;
};
```

---

3. Dependency Graphs and Circularity

Because attributes depend on each other (e.g., a synthesized attribute might depend on an inherited one), their evaluation order is critical.

3.1 Evaluation Order

We can draw a **Dependency Graph** for the attributes in a parse tree. The nodes are the attributes, and directed edges represent dependencies.

To evaluate the attributes, the compiler performs a **Topological Sort** of the dependency graph. Any evaluation order that respects this graph is valid.

3.2 The Circularity Problem

A fatal error occurs if the dependency graph contains cycles (circularity), meaning attribute A depends on B, and B depends on A.

Testing a grammar for circularity is computationally expensive (exponential time). Therefore, compilers generally restrict themselves to **Strongly Non-Circular** attribute grammars, which guarantee safe evaluation.

---

4. Drawbacks: The "Copy Rule" Problem

While attribute grammars are a powerful purely functional approach to formalize semantics, they can become cumbersome in practice.

Case Study: Cost Estimation with State

Suppose we want to estimate the execution cost of a block of expressions. Operations (`+`, `*`) have a fixed cost, but loading a variable from memory costs something *only the first time* it is encountered.

To track this state functionally, we must pass an environment down and up the tree using `before` and `after` attributes:

- `before`: The list of variables seen before evaluating this node.
- `after`: The list of variables seen after evaluating this node.

The Problem: This requires writing an enormous amount of **copy rules**. Information must be explicitly copied from the `after` attribute of one child to the `before` attribute of the next, all the way up and down the tree, even through nodes that do not care about this information. This boilerplate makes purely functional attribute grammars heavy, which is why real-world compilers often rely on ad-hoc syntax-directed translation with global state (like symbol tables) instead.

11. Practical Data-Flow Analysis

This section bridges the theoretical foundations of Data-Flow Analysis with its practical implementation within a compiler, focusing on the algorithms used to traverse the Control-Flow Graph (CFG) and gather information for optimizations.

---

1. General Procedure and Architecture

Data-flow analysis approximates the set of values at different locations in the program to assess formal correctness or to perform optimizations.

All data-flow analyses share a parametric procedure: 1. **Build the CFG**: The foundational graph representing the program. 2. **Initialize Values**: Associate a pair of initial values (`in` and `out` states) to each block. 3. **Local Update**: Define how local information is propagated from a block's neighbors (predecessors or successors) and updated internally. 4. **Global Update**: Iterate the local updates until a **fixpoint** is reached (i.e., the sets stop changing). * *Implementation note:* A naive "repeat-until-nothing-changes" loop works for simple projects, but production compilers use a **worklist approach** for better performance, processing only the blocks whose dependencies have changed.

1.1 Block Size Precision

The size of the basic blocks affects the complexity of the analysis: * **Maximal Blocks:** Contain sequences of instructions. They reduce the number of nodes but require complex internal refinement (e.g., handling a variable that is assigned multiple times within the same block). * **Minimal Blocks:** Contain a single instruction. They inherently offer maximum precision without the risk of losing information within the block, though they expand the graph size.

---

2. Defined Variables Analysis (Forward)

This analysis computes the set of variables that have been assigned a value (populated) at a specific location. It is primarily used to detect errors, such as using an undefined variable.

- **Direction:** Forward (propagates from predecessors).
- **Local Updates:**

$$Def_{in}(b) = \bigcap_{p \in pred(b)} Def_{out}(p)$$

$$Def_{out}(b) = Gen(b) \cup Def_{in}(b)$$

(where $Gen(b)$ represents the variables defined inside block b).

- **Note:** The intersection $\cap$ is used because a variable is considered safely defined *only if* it is defined along all incoming paths.

---

3. Live Variables Analysis (Backward)

This analysis computes which variables are defined and will be used later without being overwritten. It is fundamental for **Dead Store Elimination** (removing useless memory stores) and **Register Allocation** (avoiding placing dead values in scarce CPU registers).

- **Direction:** Backward (propagates from successors).
- **Local Updates:**

$$Live_{in}(b) = Gen(b) \cup (Live_{out}(b) \setminus Kill(b))$$

$$Live_{out}(b) = \bigcup_{s \in succ(b)} Live_{in}(s)$$

(where $Gen(b)$ are variables used before any assignment in b, and $Kill(b)$ are variables assigned in b).

---

4. Reaching Definitions (Forward)

This analysis computes the specific assignments (definitions) that can affect a block without being overwritten by further changes. It is crucial for **Constant Folding** and **Constant Propagation**.

- **Direction:** Forward (propagates from predecessors).
- **Local Updates:**

$$Reach_{in}(b) = \bigcup_{p \in pred(b)} Reach_{out}(p)$$

$$Reach_{out}(b) = Gen(b) \cup (Reach_{in}(b) \setminus Kill(b))$$

(where $Gen(b)$ are definitions generated in b, and $Kill(b)$ are all other definitions for variables assigned in b).

- **Identification:** To be precise, especially with maximal blocks, the analysis must track **Unique Identifiers** for each assignment rather than just the variable name or block ID, otherwise distinct assignments to the same variable cannot be differentiated.

12. Syntax-Directed Translation & IR Generation

This section explores the transition from the theoretical framework of Attribute Grammars to the practical, ad-hoc techniques used in modern compilers to perform semantic analysis and generate Intermediate Representations (IR) directly during the parsing phase.

---

1. The Realist's Alternative: Ad-hoc SDT

While Attribute Grammars (AGs) provide a clean, functional specification for semantics, they struggle with non-local information. * **The Copy Rule Problem:** Passing non-local information (like a symbol table or a global cost counter) requires an excessive number of "copy rules" to move data up and down the tree. * **Tree Construction:** Pure AGs generally require the compiler to fully build the parse tree to evaluate the attributes.

To solve these problems, compilers drop the strict functional approach and use **Ad-hoc Syntax-Directed Translation (SDT)**. * SDT associates a snippet of code (an action) with each production. * These actions can read or write from a central repository (global tables or variables), granting easy access to non-local information. * In a bottom-up parser, the corresponding snippet of code runs immediately when a reduction occurs.

---

2. Generating Intermediate Representations

During SDT, the semantic actions are typically responsible for translating the source code into an Intermediate Representation. IRs generally fall into three major categories: Structural, Linear, and Hybrid.

2.1 Structural IR: Abstract Syntax Tree (AST)

An AST retains the essential structure of the parse tree but eliminates the non-terminal nodes. It is graphically oriented and heavily used in source-to-source translators. To build an AST bottom-up, the semantic actions invoke node constructors: * For a production like `Expr -> Expr + Term`, the action would be `$$ = MakeAddNode($1, $3);`. * The pointers to these constructed nodes are passed up the parse tree as synthesized attributes.

2.2 Linear IR: Three-Address Code (ILOC)

Linear code consists of a sequence of instructions that execute in their order of appearance, acting as pseudo-code for an abstract machine. To generate linear code (like ILOC) bottom-up, the actions allocate temporary registers and emit instructions sequentially: * For a production like `Expr -> Expr + Term`, the action creates a new destination register (`$$= NextRegister();`) and emits the code (`Emit(add, $1, $3,$$);`).

---

3. Augmenting the LR(1) Parser

To execute these ad-hoc code snippets and pass attributes around without explicitly building a parse tree, the standard LR(1) skeleton parser must be modified to stash attributes directly on the parsing stack.

3.1 The 3-Item Stack

Instead of pushing just two items (the state and the grammar symbol), the augmented parser pushes **three items** for each recognized symbol: 1. The token/symbol. 2. The parser state. 3. The attribute value associated with that symbol (often denoted as `$$` for the left-hand side, and `$1, $2, ...` for the right-hand side symbols).

3.2 The Augmented Reduction Phase

When the parser decides to reduce using a production $A \to \beta$: 1. It pops $3 \times |\beta|$ items from the stack (removing the right-hand side symbols, their states, and their attributes). 2. It executes a giant `case` (or `switch`) statement based on the production number. Each case holds the code snippet for that specific production, computing the new attribute value `$$`. 3. It pushes the new left-hand side symbol A, the newly computed attribute `$$`, and the new state (determined by `GOTO[current_state, A]`) back onto the stack.

This mechanism allows the compiler to seamlessly interleave syntax verification, semantic checking, and IR generation in a single, highly efficient pass.

14. The Procedure Abstraction

This section covers the compilation challenges associated with procedure calls, focusing on how the compiler bridges the gap between static compile-time analysis and dynamic run-time execution.

---

1. Compile-Time vs. Run-Time Responsibilities

A procedure provides a clean abstraction for the programmer (independent namespaces, isolated local variables, standard interfaces), but it is the compiler's job to make this abstraction work on the physical hardware.

The compiler must generate the linkage code that manages the transition between caller and callee. This requires distinguishing between: * **Compile-Time:** The compiler determines the layout of memory, computes offsets, and emits the code to manage state. * **Run-Time:** The generated code executes, allocating memory, resolving addresses, and preserving the execution context.

---

2. Name Spaces and Scoping

Procedures define independent namespaces. A local variable declared inside a procedure hides a global variable with the same name.

2.1 Lexical vs. Dynamic Scoping

- **Lexical (Static) Scoping:** A variable name is resolved by looking at the lexical structure of the source code. If a variable is not found in the current block, the compiler looks at the immediately enclosing block, up to the global scope. This allows the compiler to determine variable bindings statically.
- **Dynamic Scoping:** A variable name is resolved by looking at the most recent active run-time binding (the call stack). This is generally slower and less predictable, and rarely used in modern low-level or systems programming (like C++).

Variables are internally mapped to a coordinate system: `<lexical_level, offset>`. * **Lexical Level:** The nesting depth of the procedure (e.g., Global = 0, Main = 1, Nested function = 2). * **Offset:** The byte offset of the variable relative to the start of its storage area.

---

3. Activation Records (Stack Frames)

Every time a procedure is invoked, a new Activation Record (AR) is created (typically pushed onto the stack). It stores all the information required to execute the procedure and return control to the caller.

3.1 Structure of an Activation Record

While specific layouts depend on the architecture and linkage conventions, a standard AR contains: 1. **Actual Parameters:** Arguments passed by the caller. 2. **Return Value:** Space allocated to pass the result back. 3. **Return Address:** The instruction pointer to resume in the caller's code. 4. **Saved Registers:** Copies of machine registers that the current procedure will overwrite but must be restored before returning. 5. **Access Link / Static Link:** A pointer to the AR of the lexically enclosing procedure (used to find non-local variables). 6. **Local Variables:** Storage for variables declared inside the procedure.

3.2 Handling Variable-Length Data

The compiler computes the static `offset` for each local variable. If a variable has a size that is unknown at compile time (e.g., dynamically sized arrays), it cannot be placed sequentially with fixed-size variables without ruining their constant offsets. * **Solution:** The compiler allocates a standard, fixed-size pointer inside the AR at a known offset. This pointer points to the actual variable-length data, which is allocated at the end of the AR (or on the heap).

---

4. Accessing Non-Local Variables

In languages that allow nested procedures, a function might access a variable declared in its parent. Since the parent's AR is somewhere on the stack, the compiler must generate code to locate it at run-time. There are two primary techniques:

4.1 Access Links (Static Links)

The AR includes a pointer to the AR of the lexically enclosing procedure. * **Mechanism:** If a procedure at Level 3 needs a variable from Level 1, the generated code must traverse the access links backward: `current_AR -> access_link -> access_link`. * **Performance:** Finding the variable takes time proportional to the difference in lexical levels: $O(\Delta\text{levels})$. * **Maintenance:** When a procedure calls another, it passes the appropriate access link as a hidden parameter.

4.2 The Display Array

Instead of chaining pointers through the stack, the system maintains a global array of pointers called the **Display**. * **Mechanism:** `Display[i]` always points to the currently active AR for lexical level `i`. To access a variable at Level 1, the code directly loads `Display[1]` and adds the variable's offset. * **Performance:** Variable access is strictly $O(1)$ constant time. * **Maintenance:** Calling a procedure requires updating the Display array. Returning requires restoring the previous Display state. The overhead is shifted from *variable access* to *procedure invocation*.

---

5. Linkage Conventions (Caller vs. Callee)

The linkage convention dictates the contract between the caller and the callee.

Register Saving Responsibilities

Registers are a shared resource. Who is responsible for saving them to the AR? * **Caller-Saved:** The calling procedure saves registers it cares about because it assumes the callee will overwrite them. Advantage: The caller only saves what is actively live. * **Callee-Saved:** The called procedure saves the registers it plans to use and restores them before returning. Advantage: If the callee doesn't use a register, no memory traffic is wasted.

Modern compilers (especially for performance-critical languages like C++) use a hybrid approach dictated by the Application Binary Interface (ABI), statically assigning some registers as caller-saved and others as callee-saved to minimize memory operations.

```
// pseudo-code illustrating caller/callee save convention
void caller() {
    // save caller-saved registers (if live)
    // push parameters
    // call callee
    callee();
    // restore caller-saved registers
```

```
}

void callee() {
    // prologue: push callee-saved registers to AR
    // execute body
    // epilogue: pop callee-saved registers from AR
    // return
}
```

15. Code Generation & Shape Optimization

This section explores the transition from the intermediate representation (IR) to target-machine code. The compiler must make crucial choices regarding how language constructs are shaped into instructions, how variables are accessed, and how memory is managed, significantly impacting the performance of the final executable.

---

1. The Importance of Code Shape

"Code shape" refers to the specific sequence of instructions chosen to implement a high-level construct. The compiler often has multiple valid ways to translate a single statement, but these choices are not created equal regarding efficiency.

For instance, consider translating a `switch` or `case` statement. The compiler could choose: 1. **Sequential `if-else` (Linear Search):** Checking each condition sequentially. The execution cost depends heavily on which case is triggered. 2. **Binary Search:** More efficient if the cases are sparse but ordered. 3. **Jump Table:** Creating an array of addresses and jumping directly to the correct case in $O(1)$ time.

The "best" choice depends on the specific context (e.g., the number of cases and their density). The compiler's back-end (optimizer and code generator) must make these choices because a purely mechanical translation lacks the context to select the optimal shape.

1.1 Context-Dependent Optimization

Consider the expression `x + y + z`. Because addition is commutative and associative, the compiler could generate: 1. `(x + y) + z` 2. `x + (y + z)` 3. `(x + z) + y`

Which is best? It depends on the context. * If the compiler knows statically that `x = 2` and `z = 3`, it can precompute `x + z = 5` (Constant Folding), reducing the expression to `5 + y`. * If `y + z` was already computed in a previous statement, the compiler can reuse that result (Common Subexpression Elimination), saving instructions.

---

2. Memory Models: Register-to-Register

Before generating code, the compiler must adopt a memory model. The most common model for modern architectures (especially RISC) is the **Register-to-Register** model.

- **Concept:** The compiler assumes an infinite number of virtual registers. It attempts to keep all values in registers for as long as possible.
- **The Process:** During the initial code generation, every new value gets a brand-new virtual register.
- **The Catch:** Physical machines have a limited number of actual registers.
- **The Solution:** A later phase called **Register Allocation** maps the infinite virtual registers to the finite physical ones. If there aren't enough physical registers, the allocator inserts "spill" code (stores and loads) to temporarily move values to memory.

(Contrast this with the Memory-to-Memory model, where values are assumed to live in memory and are only loaded into registers right before an operation, requiring the optimizer to remove redundant memory accesses).

---

3. Tree-Walk Code Generation

To generate linear code (like ILOC) from an Abstract Syntax Tree (AST), the compiler performs a **post-order traversal** (tree-walk). * It visits the children before evaluating the parent node.

3.1 The Translation Routine

The core of this process is a recursive function `expr(node)` that emits instructions and returns the register containing the result.

Algorithm logic: 1. **If node is an Identifier (Variable):** * Retrieve the base address (the pointer to the Activation Record) into a register. * Retrieve the variable's offset within that record into another register. * Emit a `load` instruction using base + offset, putting the variable's value into a new result register. 2. **If node is a Number (Constant):** * Emit a `load immediate` instruction to put the constant value into a new result register. 3. **If node is an Operator (+, -, *, /):** * Recursively call `expr()` on the left child, getting its result register. * Recursively call `expr()` on the right child, getting its result register. * Emit the arithmetic instruction using the two child registers, putting the final result into a new register.

3.2 The Impact of Evaluation Order

A naive post-order traversal (always evaluating the left child first, then the right) is simple but not always optimal regarding register usage.

If an expression tree is unbalanced, always evaluating the left side first might require keeping many intermediate results alive in registers simultaneously. * **Sethi-Ullman Algorithm (Concept):** By strategically alternating the evaluation order—evaluating the child subtree that requires *more* registers first—the compiler can minimize the peak number of registers needed simultaneously. If the peak register usage stays below the physical machine limit, the compiler completely avoids expensive memory spills.

16. Laboratory (LLVM Infrastructure & IR)

This section covers LLVM, a modular and language-agnostic compiler infrastructure. It details the architecture of the LLVM pipeline and the core concepts of its Intermediate Representation (IR), which is the target for the code generation phase of the laboratory project.

---

1. The LLVM Compiler Infrastructure

LLVM is a collection of modular and reusable compiler and toolchain technologies. Unlike older monolithic compilers, LLVM strictly separates the compilation process into three distinct phases:

1. **Front-End:** Translates a specific high-level language (C++, Rust, Swift, etc.) into LLVM IR. Examples include Clang (for C/C++) and rustc (for Rust).
2. **Middle-End (Optimizer):** Takes the LLVM IR, analyzes it, and applies a series of target-independent optimization passes to improve performance.
3. **Back-End:** Translates the optimized LLVM IR into specific machine code for target architectures (x86-64, ARM, RISC-V, etc.).

This modularity allows developers to create a new programming language simply by writing a front-end that emits LLVM IR, instantly gaining access to world-class optimizers and cross-platform machine code generation.

---

2. LLVM Intermediate Representation (IR)

The core of the LLVM framework is its Intermediate Representation. It acts as the universal language shared between all compilation phases.

2.1 Properties of LLVM IR

- **RISC-like:** It uses simple, low-level instructions (arithmetic, memory transfers, branches).
- **Infinite Virtual Registers:** It assumes the machine has an unlimited number of registers, leaving the physical register allocation to the back-end.
- **Strongly Typed:** Every instruction and register must explicitly declare its type. There are no implicit type conversions; casting requires explicit instructions.
- **Three Equivalent Formats:**
  1. *In-memory:* Binary format used internally during compilation.
  2. *Bitcode:* On-disk binary format (`.bc`) for fast loading and linking.
  3. *Assembly:* Human-readable textual format (`.ll`) used for debugging and development.

---

3. Hierarchical Structure

LLVM IR organizes code into a strict hierarchy:

3.1 Modules

The Module is the top-level container (conceptually similar to a translation unit or a `.c` file). It encapsulates: * Global variables and constants. * Function declarations (external functions to be linked later) and definitions. * Symbol tables and metadata.

3.2 Functions

Functions contain a signature (return type and arguments) and a body. The body is composed of a list of Basic Blocks.

3.3 Basic Blocks

A Basic Block is a sequence of instructions that execute linearly. * It must have a single entry point (the label at the top). * It must end with exactly one **terminator instruction** (e.g., a branch `br` or return `ret`), dictating the control flow to the next block.

---

4. The Type System

LLVM requires explicit typing to enable aggressive optimizations and correct memory alignment.

- **Primitive Types:** Integers of arbitrary bit width (`i1` for booleans, `i8`, `i32`, `i64`), and floating-point types (`float`, `double`).
- **Pointers:** Memory addresses, historically typed (e.g., `i32*`), but increasingly represented as opaque `ptr` in newer LLVM versions.
- **Aggregates:** * *Arrays:* Sequential elements of the same type (e.g., `[4 x i32]`).
  - *Structs:* Collections of mixed types in memory (e.g., `{i32, float}`).
- **Vectors:** Specifically used for SIMD (Single Instruction, Multiple Data) hardware operations (e.g., `<4 x i32>`). A single instruction can operate on all vector elements simultaneously.

---

5. Static Single Assignment (SSA) and Phi Nodes

LLVM IR strictly adheres to **Static Single Assignment (SSA)** form. * Every virtual register (denoted with a `%` prefix, e.g., `%1`, `%val`) is assigned a value exactly once. * Registers cannot be mutated or reassigned.

5.1 The `phi` Instruction

Because registers cannot be reassigned, control flow merging (like an `if-else` rejoining) requires a way to select the correct value based on which path was taken. This is solved using the ϕ (phi) instruction.

```
; Example of a phi node selecting a value based on the predecessor block
%res = phi i32 [ %val_from_if, %if_block ], [ %val_from_else, %else_block ]
```

The `phi` node looks at which basic block executed immediately before the current one and assigns `%res` the corresponding value. *Note: `phi` instructions must always be placed at the very top of a basic block.*

5.2 The "Alloca" Workaround

Generating pure SSA form with `phi` nodes directly from an AST is complex. Compilers often use a trick:

1. Allocate local variables on the stack using `alloca`.
2. Mutate them using `store` and read them using `load`.
3. Pass the resulting IR to LLVM's `mem2reg` optimization pass, which automatically promotes the stack memory into proper SSA registers and inserts `phi` nodes for you.

---

6. Core Instructions Overview

- **Memory Operations:** LLVM uses a load/store architecture.
  - `alloca`: Allocates memory on the stack frame of the current function.
  - `load` / `store`: Reads from or writes to a memory pointer.
  - `getelementptr` (GEP): Computes the memory address of a sub-element (like an array index or a struct field) *without* accessing memory. It purely calculates the pointer offset.
- **Control Flow:**
  - `br label %dest`: Unconditional branch.
  - `br i1 %cond, label %true_dest, label %false_dest`: Conditional branch.
  - `ret i32 %val`: Return from a function.
- **Arithmetic & Logic:** `add`, `sub`, `mul`, `sdiv` (signed division), `icmp` (integer comparison), `and`, `or`, `shl` (shift left).
- **Intrinsics:** Predefined compiler functions (starting with `@llvm.`) that map directly to complex hardware instructions or standard library routines (e.g., `@llvm.memcpy`).

---

7. The Pass Manager

Transformations in the middle-end are organized into "passes".

- **Analysis Passes:** Gather information (e.g., control-flow dominance, variable liveness) without mutating the IR.
- **Transform Passes:** Mutate the IR to optimize it (e.g., constant folding, dead code elimination).

The **Pass Manager** orchestrates these passes, automatically scheduling them based on their dependencies, caching analysis results, and re-running them if a transformation invalidates previous data.

17. Code Generation to LLVM IR

This section details the practical translation of a high-level imperative language (like MiniImp) into LLVM Intermediate Representation, bridging the gap between abstract syntax and machine-level execution.

---

1. Deconstructing Expressions

LLVM IR instructions are simple and strictly limited in the number of operands. Complex arithmetic or boolean expressions cannot be represented in a single instruction.

- **Flattening:** A complex expression like `x = 2 + y * z` must be broken down into a sequence of fundamental operations.
- **Fresh Registers:** Because LLVM IR operates under the Register-to-Register model, every intermediate step of a computation must be stored in a newly generated, unique virtual register.

```
// Pseudocode translation of: x = 2 + y * z
%temp1 = load y
%temp2 = load z
%temp3 = mul %temp1, %temp2
%temp4 = add 2, %temp3
store %temp4 into x
```

---

2. Control Flow and Basic Blocks

Control flow constructs (like `if-else` and `while`) must be translated into a flat sequence of Basic Blocks connected by explicit branch instructions.

2.1 Mapping the CFG

The conceptual Control-Flow Graph previously built is now mapped directly to LLVM IR blocks:

- Every node in the CFG becomes an LLVM Basic Block starting with a fresh label (e.g., `entry:`, `if_true:`, `if_false:`, `merge:`).
- Every block **must** end with a terminator instruction. If the block naturally falls through to the next, an unconditional branch (`br label %next_block`) must be explicitly inserted.

2.2 The Pragmatics of Block Size

The compiler designer must choose how to group statements into blocks:

- **Minimal Blocks:** Creating a new basic block for every single statement is easy to implement but generates excessive jumping. This makes unoptimized code extremely slow and hinders the debugging experience.
- **Maximal Blocks:** Grouping as many sequential statements as possible into a single block improves baseline performance and readability, making the compiler more pragmatic and user-friendly before optimizations are even applied.

---

3. Surviving the SSA Form Requirement

LLVM IR enforces Static Single Assignment (SSA) form, meaning a virtual register can only be assigned a value exactly once. This creates challenges for imperative variables that are mutated over time (e.g., `x = 1; x = x + 1;`).

3.1 The Phi Node Solution

When a variable is assigned different values in different branches (e.g., inside an `if` and an `else` block), the compiler must use a ϕ (`phi`) node at the merge point to select the correct value based on the execution path.

- **Constraint:** `phi` nodes must be placed at the absolute beginning of the basic block, before any other instruction.

3.2 The "Alloca" Memory Trick (Cheating SSA)

Generating perfect SSA form and computing the correct placement of `phi` nodes manually during AST traversal is complex (requiring Dominator Tree analysis).

Instead, compiler front-ends commonly use the following strategy to bypass manual SSA generation:

1. **Allocate on Stack:** For every declared variable, emit an `alloca` instruction to reserve memory on the stack.
2. **Load and Store:** Never keep the variable's value in an SSA register across statements. Whenever the variable is read, emit a `load`. Whenever it is updated, emit a `store`.
3. **Delegate to LLVM:** Produce this inefficient, memory-heavy IR. Then, invoke the LLVM optimizer with the `mem2reg` pass. LLVM will automatically analyze the stack allocations, promote them to SSA registers, and perfectly insert all necessary `phi` nodes.

---

4. Compilation and Tooling

To manipulate and compile the generated IR, the LLVM infrastructure provides several command-line tools:

- `opt`: The LLVM optimizer. It takes the unoptimized IR and applies passes.
  - Example: `opt -passes=mem2reg input.ll -S -o optimized.ll` (The `-S` flag ensures the output remains in human-readable textual format rather than binary bitcode).
- `llc`: The LLVM static compiler. It translates the optimized IR into target-specific assembly or machine code (e.g., an `.s` or `.o` file).
- **Compiler Explorer (godbolt.org):** A crucial web-based visualization tool. It allows developers to write code, see the generated LLVM IR, and inspect the exact sequence of optimization passes (the "pipeline") to understand how the IR is transformed into efficient machine code.

18. Code Shape and Advanced Optimization Strategies

This section details the strategies compilers use to map complex language constructs to machine-level instructions. The choices made during this translation (the "code shape") directly dictate the efficiency of the final executable, particularly concerning memory access, loops, and branching.

---

1. Memory Models and Evaluation Order

The translation from the Abstract Syntax Tree (AST) to the Intermediate Representation (IR) is heavily influenced by the chosen memory model.

- **Register-to-Register Model:** The compiler assumes an infinite number of virtual registers. It attempts to keep every computed value in a register for as long as possible. The duty of mapping these virtual registers to physical hardware and managing "spills" (saving to memory) is deferred to the Register Allocation phase.
- **Memory-to-Memory Model:** The compiler assumes all values reside in memory, generating `load` and `store` instructions for every operation. Optimization phases must later identify which memory accesses can be safely replaced with registers.

When walking the AST to generate code, the **evaluation order** of expressions is critical. Evaluating the left child before the right child (or vice-versa) yields the same final result but can drastically change the number of intermediate registers required. A smarter evaluation order minimizes peak register pressure, reducing the need for costly memory spills later on.

---

2. The Ambiguity Problem (Aliasing)

Under the register-to-register model, the compiler wants to keep variables in registers. However, this is unsafe if a value is **ambiguous**—meaning the same memory location can be accessed via multiple names (aliasing).

The Danger of Aliasing

Consider two variables, `a` and `b`, passed by reference or accessed via pointers. If `a` and `b` point to the same memory address, they alias each other. 1. The compiler loads `a` into register `r1` and `b` into register `r2`. 2. An assignment modifies `b` (updating `r2` and eventually memory). 3. The compiler later reads `a` from `r1`.

If `a` and `b` are aliased, the value in `r1` is now stale, leading to incorrect execution. Because the compiler cannot always determine at compile-time whether two pointers overlap, it must act conservatively: **ambiguous values cannot be safely kept in registers** across potential updates. They must be loaded from memory before use and stored immediately after.

To mitigate this, compilers rely on **Alias Analysis** (or sharing analysis). In languages like C++, features like the `__restrict` keyword explicitly tell the compiler that a pointer has no aliases, allowing it to aggressively keep the value in a register.

---

3. Arrays, Matrices, and Loop Optimizations

Translating data structures requires computing precise memory offsets. Array and matrix operations are frequently nested inside loops, making their optimization critical for performance.

3.1 Offset Calculation

For a 1D array with a known lower bound (`low`), accessing `A[i]` requires computing: `address = base_address(A) + (i - low) * element_size`

For multi-dimensional matrices, the layout matters: * **Row-Major Order (C/C++):** Elements of the same row are contiguous in memory. * **Column-Major Order (Fortran):** Elements of the same column are contiguous. The access formula must account for the sizes of all preceding dimensions.

3.2 Loop Optimizations

Since offset calculations involve multiplications and additions, repeatedly executing them inside a loop is expensive. 1. **Loop Invariant Code Motion (Hoisting):** If an array access involves multiple dimensions (e.g., `Matrix[i][j]`), the offset computation for `i` does not change during the inner loop over `j`. The compiler "hoists" the invariant calculation of `i` outside the inner loop, computing it only once. 2. **Strength Reduction:** Multiplications inside a loop (e.g., `offset = i * element_size`) can be replaced by cheaper additions. The compiler initializes a pointer before the loop and simply adds `element_size` to it at the end of each iteration.

---

4. String Representation and Hardware Dependencies

Strings can be represented in two primary ways, each dictating a different code shape for string operations: 1. **Explicit Length (Length-Prefixed):** The string stores its length metadata followed by the characters. Operations like bounds checking and length querying are $O(1)$, but the structure requires more memory and strict synchronization between the length header and the payload. 2. **Null-Terminated (C-style):** The string ends with a specific sentinel character (`\0`). Length querying is $O(N)$, but memory overhead is minimal.

The generated code for manipulating strings (e.g., copying or modifying a single character) is heavily hardware-dependent. If the architecture only supports word-aligned memory operations (e.g., copying 4 or 8 bytes at a time), modifying a single character requires loading the entire word, applying a **bitwise mask** to isolate and replace the specific byte, and storing the word back. If the architecture has byte-level instructions, the compiler can emit a direct byte-store.

---

5. Translating Boolean Expressions and Control Flow

Boolean expressions (evaluating to true/false) drive the control flow of the program. The compiler can translate them using different hardware mechanisms.

5.1 Representation Strategies

- **Explicit Values:** The comparison instruction explicitly writes a numerical `1` (true) or `0` (false) into a general-purpose register.
- **Condition Codes:** The comparison instruction does not write to a general register. Instead, it sets hidden hardware flags (Zero Flag, Sign Flag, Overflow Flag). Subsequent conditional branch instructions (e.g., `jump_if_equal`) read these flags to alter the program counter.

5.2 Eliminating Branches

Hardware pipelines suffer severe performance penalties when conditional branches are mispredicted. A major optimization goal is to eliminate branches entirely for small operations.

Instead of translating an `if-else` statement into multiple basic blocks with jumps, modern compilers use **Predicated Execution** or **Conditional Moves** (`cmov`).

```
// source
int x = (a < b) ? a : b;

// conditional move shape (no branches)
// t1 = a
// t2 = b
// cmp a, b
```

```
// cmov_lt x, t1   // move t1 into x if less-than flag is set
// cmov_ge x, t2   // move t2 into x if greater-or-equal flag is set
```

By evaluating both potential outcomes into temporary registers and using conditional moves to commit the correct one, the compiler avoids branch instructions completely, maintaining a steady instruction pipeline.

19. Scopes and Value Numbering

This section delves into the optimizer, exploring how the compiler transforms the Intermediate Representation (IR) to improve execution speed, memory footprint, and power consumption without altering the program's observable semantics.

---

1. Scopes of Optimization

Optimizations can be applied at different levels of the program's structure. The broader the scope, the more opportunities the compiler has, but the more computationally expensive the analysis becomes.

1. **Local:** Operates within a single Basic Block (straight-line code). Strong guarantees exist because there is no control flow to disrupt the state.
2. **Superlocal:** Operates over an **Extended Basic Block (EBB)**—a sequence of blocks where each block (except the first) has exactly one predecessor. It allows propagating information down a specific linear path.
3. **Regional:** Operates over broader CFG regions, such as loops.
4. **Global:** Operates over an entire procedure or function.
5. **Interprocedural:** Operates across the whole program (multiple functions/files).

---

2. Translating Branches: Predication vs. Jumping

When the compiler encounters conditional statements, it must decide the most efficient hardware translation.

- **Standard Branching:** The compiler generates jump instructions based on condition codes. This is safe but incurs heavy hardware penalties if the CPU mispredicts the branch.
- **Predicated Execution / Conditional Moves:** The compiler evaluates *both* branches of the condition sequentially and uses a conditional move (`cmov`) to select the correct final value.

Trade-offs of Predication: Predication eliminates costly branch instructions. However, it is only effective if: 1. Both execution paths are relatively short. If one path takes 100 cycles and the other takes 1, forcing the CPU to compute both wastes time. 2. The paths have no side effects (like memory stores or exceptions) that cannot be easily rolled back if the condition evaluates to false.

---

3. Local Value Numbering (LVN)

Value Numbering is a powerful local optimization technique used primarily for **Common Subexpression Elimination (CSE)**. The goal is to avoid recomputing expressions that have already been evaluated.

3.1 The Algorithm

The compiler traverses the instructions in a basic block and assigns a unique integer (a "Value Number") to every distinct computed value. It maintains a Hash Table mapping `(Operator, ValueNum_Left, ValueNum_Right)` to a `Result_ValueNum`.

```
// Example execution of LVN
1. a = b + c    // Hash(+, VN(b), VN(c)) is new. Assign VN 1 to 'a'.
2. d = b + c    // Hash(+, VN(b), VN(c)) matches VN 1!
                // Optimization: Replace with 'd = a'
3. e = a - d    // Hash(-, VN 1, VN 1).
```

3.2 Extensions and Algebraic Identities

LVN can be augmented to catch more redundancies:

- **Commutativity:** For commutative operators like `+` or `*`, the compiler sorts the operand value numbers before hashing (e.g., `Hash(+, 2, 1)` becomes `Hash(+, 1, 2)`) to recognize that `b + c` is the same as `c + b`.
- **Constant Folding:** If the operands are constants, compute the result immediately at compile-time.
- **Algebraic Identities:** Hardcode rules to simplify expressions dynamically, such as $x + 0 = x$, $x * 1 = x$, or $x - x = 0$.

3.3 The Naming Problem and SSA

A major limitation of standard LVN is variable reassignment.

If the code does `a = x + y`, followed later by `x = 10`, and then `b = x + y`, the name `x` means something different the second time. The compiler must carefully invalidate previous value numbers associated with `x`.

This is why **Static Single Assignment (SSA) form** is so critical. In SSA, variables are never reassigned (`x1`, `x2`). This allows the LVN algorithm to safely rely on variable names as pure representations of values, dramatically simplifying the optimization pass.

---

4. Superlocal Value Numbering (SVN)

Superlocal Value Numbering extends LVN beyond a single basic block by applying it to Extended Basic Blocks (EBBs).

Since an EBB is a path where control flow does not merge from unknown locations, the compiler can take the hash table generated at the end of the first block and pass it directly as the starting state for the next block in the path. This allows finding common subexpressions across block boundaries, provided the execution path strictly dictates that the first block must have been executed to reach the second.

---

5. Loop Unrolling

Loops carry inherent overhead at runtime: incrementing the counter, evaluating the condition, and branching back to the top.

Loop Unrolling mitigates this overhead by replicating the loop body multiple times (the "unroll factor").

5.1 Benefits

- If a loop is unrolled by a factor of 4, the branch and condition overhead is reduced by 75%.
- It creates larger basic blocks, giving local optimizers (like LVN and instruction schedulers) a much larger window of code to work with, unlocking new optimization opportunities.
- It can eliminate redundant end-of-loop copy operations by shifting variable names across the unrolled iterations.

5.2 Drawbacks and Edge Cases

- **Code Bloat:** Unrolling increases the size of the compiled executable, which can put negative pressure on the CPU's Instruction Cache.
- **Unknown Bounds:** If the compiler does not know the exact number of loop iterations statically, it must generate a "Guard Loop" or "Cleanup Loop" to handle the remaining iterations if the total count is not perfectly divisible by the unroll factor.

20. Data-Flow Analysis & Liveness Analysis

This section delves into the formal mechanisms of Data-Flow Analysis, a cornerstone of static analysis used in the compiler's middle-end to gather information about how data propagates through a program. This information is strictly required to safely apply optimizations and perform register allocation.

---

1. The Data-Flow Framework

Data-flow analysis operates on the Control-Flow Graph (CFG), where each node represents a basic block (or a single statement). The analysis extracts properties by observing how information changes as it passes through these nodes.

1.1 IN and OUT Sets

For every node n in the CFG, we define two mathematical sets to track the flow of information: * $IN[n]$**:** The state of information immediately *before* entering node n. * $OUT[n]$**:** The state of information immediately *after* exiting node n.

The difference between IN and OUT represents the **transfer function** (the effect that the specific code inside the node has on the program state).

1.2 Dimensions of Analysis

Data-flow problems are categorized by two main dimensions: 1. **Direction:** * **Forward:** Information flows along the execution paths (from predecessors to successors). * **Backward:** Information flows against the execution paths (from successors to predecessors). 2. **Confluence (May vs. Must):** * **May (Possible) Analysis:** We want to know if a property *might* hold along *at least one* path. When control flow merges, we use the **Union ($\cup$)** operator. * **Must (Definite) Analysis:** We want to know if a property holds along *all* possible paths. When control flow merges, we use the **Intersection ($\cap$)** operator.

---

2. Liveness Analysis

Liveness Analysis is a quintessential **Backward, May** data-flow problem.

- **Definition:** A variable is considered "live" at a specific program point if its current value *may* be read (used) later in the execution before it is overwritten (redefined).
- **Primary Objective: Register Allocation**. If two variables are never live at the same time (their live ranges do not intersect), the compiler can safely assign both of them to the same physical CPU register, minimizing memory spills.

2.1 Local Sets: Def and Use

To compute liveness, the compiler first calculates two local properties for each node n: * $Use[n]$**:** The set of variables that are read inside the node *before* any assignment happens within that same node. * $Def[n]$**:** The set of variables that are assigned a new value (written) inside the node.

2.2 The Flow Equations

Because liveness looks into the future to see if a variable will be used, information must flow **backwards** from the end of the program to the beginning.

1. **The OUT Equation:** The variables live upon exiting a node are exactly the variables live upon entering any of its succeeding nodes. Since this is a "May" analysis, we union the inputs of the successors.

$$OUT[n] = \bigcup_{s \in succ(n)} IN[s]$$

2. **The IN Equation:** A variable is live upon entering a node if either:
   a) It is explicitly used inside this node ($Use[n]$).
   b) It is live upon exiting the node ($OUT[n]$) AND it is not overwritten inside this node ($-Def[n]$).

$$IN[n] = Use[n] \cup (OUT[n] \setminus Def[n])$$

---

3. Safe Over-Approximation

Static analysis works on the CFG without executing the code. Because the compiler cannot predict the outcome of dynamic branches (like `if (x < 0)`), it must assume that **all edges in the CFG are feasible execution paths**.

- **Unfeasible Paths:** In reality, some paths might be logically impossible (e.g., entering an `if (true)` block and then taking the `false` branch of a subsequent identical condition).
- **Over-approximation:** Because the analysis unions the liveness from *all* structural paths, it will inevitably claim that a variable is live in places where, at runtime, it actually is not.
- **Safety:** This over-approximation is structurally **safe**. In compiling, it is acceptable to have a *false positive* (keeping a dead variable in a register, slightly degrading performance). It is disastrous to have a *false negative* (discarding a variable from a register when it will actually be used later, breaking the program's semantics). Liveness analysis guarantees no false negatives.

---

4. Fixpoint Theorem and Iterative Computation

Because control-flow graphs often contain cycles (loops), the IN and OUT equations are mutually recursive. Node A depends on Node B, which depends on Node C, which loops back to Node A.

To solve this, the compiler relies on the **Fixpoint Theorem** over finite domains:

1. **The Domain:** The mathematical domain is the powerset of all variables in the program $\mathcal{P}(vars)$, ordered by set inclusion ($\subseteq$).
2. **Initialization:** Since we are computing a "May" analysis (Union-based), we start at the bottom of the lattice. All IN and OUT sets are initialized to the empty set $\emptyset$.
3. **Monotonic Iteration:** The compiler repeatedly applies the IN and OUT equations to all nodes. Because the transfer functions only use Unions and set subtractions against constant local sets, the IN and OUT sets can only grow or remain the same across iterations; they can never shrink (Monotonicity).
4. **Convergence:** Since the total number of variables is finite, the sets cannot grow forever. Eventually, an iteration will complete where no elements are added to any IN or OUT set. This stable state is the **Least Fixpoint**, representing the tightest safe approximation of liveness for the program.

21. Reaching Definitions Analysis

This section continues the exploration of Data-Flow Analysis by introducing a new property: Reaching Definitions. While Liveness Analysis looks into the future to see if a variable will be used, Reaching Definitions looks into the past to determine where a variable's current value was assigned.

---

1. What is a Reaching Definition?

A **definition** d is an assignment to a variable (e.g., $d : x = y + z$). A definition d **reaches** a point p in the program if: 1. There exists at least one execution path from d to p. 2. The variable assigned at d is not "killed" (overwritten by another assignment) anywhere along that path.

- **Analysis Type:** Reaching Definitions is a **Forward, May** analysis. Information flows in the direction of execution, and we want to know if a definition *might* reach a point.

1.1 Use Cases for the Compiler

- **Constant Propagation:** If the only definition of x that reaches a point p is $x = 5$, the compiler can safely replace the use of x at point p with the constant `5`.
- **Undefined Variables Check:** To model this, the compiler creates a special "dummy" definition for every variable at the very beginning of the program (representing the uninitialized state, often denoted as `?`). If the dummy definition `?` for variable x reaches a point where x is used, it means the variable might be used uninitialized.
- **Loop Invariant Code Motion:** To safely move an instruction $x = y + z$ outside of a loop, the compiler must prove that the definitions of y and z that reach the instruction inside the loop are exactly the same definitions that reach the point outside the loop. If they change, the instruction depends on the loop's execution and cannot be moved.

---

2. The Mathematical Domain

Unlike Liveness Analysis, where the domain was simply sets of variables, Reaching Definitions requires tracking *where* those variables were defined.

- **Domain Elements:** The domain consists of sets of pairs `(variable, definition_point)`. For example, `(x, node_2)` means "the definition of x that occurred at node 2".
- **The Dummy Definition:** We include `(x, ?)` to represent that x is uninitialized.

---

3. The Flow Equations

For each node n, we define two local sets: * $Gen[n]$**:** The set of definitions generated within node n. If node 5 is $x = a + b$, then $Gen[5] = \{(x, 5)\}$. * $Kill[n]$**:** The set of all other definitions in the program that assign to the same variable. In the previous example, $Kill[5]$ contains all pairs `(x, d)` where $d \neq 5$.

The global flow equations for a **Forward, May** analysis are:

1. **The IN Equation:** The definitions reaching the entry of a node are the union of the definitions reaching the exits of all its predecessors.

$$IN[n] = \bigcup_{p \in pred(n)} OUT[p]$$

2. **The OUT Equation:** The definitions reaching the exit of a node are those generated within the node, plus those that entered the node and were not killed by it.

$$OUT[n] = Gen[n] \cup (IN[n] \setminus Kill[n])$$

3.1 Initial Node Special Case

For the very first node in the program (the entry node), the IN set is initialized with the dummy definitions for all variables:

$$IN[entry] = \{(v, ?) \mid \forall v \in Vars\}$$

---

4. Solving the Equations

Just like Liveness Analysis, we use the Fixpoint Theorem.

1. Initialize $OUT[n] = \emptyset$ for all nodes (except potentially the dummy initialization for the entry node).
2. Iterate through the nodes, applying the IN and OUT equations.
3. Because the domain is finite and the transfer function is monotonic (we only use unions and set subtractions against constant sets), the sets will grow monotonically.
4. Stop when a full iteration produces no changes to any IN or OUT set. This is the **Least Fixpoint**.

4.1 Iteration Strategy

While you can iterate through the nodes in any order (e.g., $1, 2, 3, 4 \ldots$) and eventually reach the fixpoint, the order matters for performance. * Since Reaching Definitions is a **Forward** analysis, visiting the nodes in topological order (following the flow of execution from top to bottom) propagates information much faster, reducing the total number of iterations required to converge.

99. Laboratory

This document serves as the comprehensive guide for the Compilation Techniques laboratory project. The objective is to build a compiler that includes evaluation, static analysis, optimization, and code generation targeting LLVM IR.

The project is divided into two distinct languages: 1. **MiniImp:** A minimal imperative language focusing on control flow, data flow analysis, and optimization. 2. **MiniFun:** A minimal functional language focusing on environments, closures, and a polymorphic type system.

Project Submission and Requirements

The final submission requires two main components: 1. **Codebase:** The project code must be clean, commented, and packed into a single zip file following a specific naming convention. You can use multiple modules or files, but they must be provided together. 2. **Report & Execution Instructions:** A document containing your design ideas and clear execution instructions. If the program cannot be executed or tested by the instructor, it will not be debugged for you. Ensure edge cases (e.g., merging multiple basic blocks) are handled and documented.

Fragment 1: Language Semantics

The first milestone requires implementing the operational semantics for both languages. Each language is defined by its syntax (grammar) and semantics (meaning, via deduction systems).

MiniImp Semantics

MiniImp operates by reading and updating a memory σ, which is a partial function mapping locations (variables) to values (integers). * **Deadlocks:** A program fails if it reaches an erroneous state where the semantics are undefined. In MiniImp, this happens specifically when a variable is undefined (e.g., trying to read `y` when it has no value in σ). * **Non-termination:** The `while` loop construct can cause non-termination, resulting in an infinite derivation tree.

MiniFun Semantics

MiniFun requires managing lexical scopes and function evaluations. * **Closures:** When a function is evaluated, the semantics must capture the environment at that exact time. * Standard function `fun x => t` produces a closure `(x, t, \rho)`. * Recursive function `letfun f x = t` produces a closure that also stores its own name: `(f, x, t, \rho)`.

Fragment 4: Type System (Algorithm W)

Note: Completing this fragment is not mandatory to pass the exam, but it is required to achieve the maximum grade.

This fragment implements static analysis for MiniFun to prove properties without execution, avoiding runtime errors such as adding a boolean to an integer. It relies on propagating constraining information through the syntax tree.

Implementation Requirements

1. Define data structures for **monotypes**, **polytypes**, and **substitutions**.
2. Implement the core type-inference functions: `inst`, `gener`, and `unify`.
3. Write a `typechecking` function for MiniFun.

Algorithm W: `if-then-else` (ITE) Step-by-Step

The order of operations during type checking strictly matters. For an ITE construct: 1. Check the first argument (condition). 2. Update the context with the substitution and check the second argument (then branch). 3. Update the context with the resulting substitution and check the third argument (else branch). 4. Unify the first argument's type with `bool`. 5. Unify the types of the second and third arguments.

Fragment 5: Control-Flow Graph (CFG)

Prerequisite: Fragment 1.

This fragment involves building the Control-Flow Graph for MiniImp, a crucial intermediate step for Data Flow Analysis and Compilation.

CFG Structure

- **Nodes:** Basic blocks (sequences of simple, straight-line statements).
- **Edges:** Possible control-flow paths between the basic blocks.
- **Program Level:** Since MiniImp lacks procedures, the CFG of the entire program is simply the CFG of the main command block `c`.

Translation Rules

- **Simple Commands:** Assignments (`x := e`) or `skip` generate a graph with a single node, one entry, and one exit.
- **Sequence (`c1; c2`):** Generate graphs for `c1` and `c2`. Connect the final node(s) of `c1` to the initial node of `c2`.
- **Conditional (`if b then c1 else c2`):**
  - Create a node for `b?`.
  - True edge goes to the start of `c1`. False edge goes to the start of `c2`.
  - Both branches merge into a common `skip` node at the end to unify the control flow.
- **Looping (`while b do c`):**
  - Create a node for `b?`.
  - True edge goes to the start of the loop body `c`.
  - The end of `c` loops back to `b?`.
  - False edge connects to a `skip` node representing the loop exit.

```
// pseudocode for cfg generation of a sequence
function generate_sequence_cfg(cmd1, cmd2):
    cfg1 = generate_cfg(cmd1)
    cfg2 = generate_cfg(cmd2)

    // link the exit points of the first block to the entry of the second
    for each exit_node in cfg1.exit_nodes:
        add_edge(exit_node, cfg2.entry_node)

    return build_graph(cfg1.entry_node, cfg2.exit_nodes)
```

Fragment 6: Data-Flow Analysis

Prerequisite: Fragment 5 (CFG Generation).

This fragment implements the core data-flow analysis algorithms over the previously constructed CFG for MiniImp, laying the groundwork for optimizations.

Implementation Requirements

1. **Extend CFGs:** Modify the CFG data structures to support annotations on blocks (to store the `in` and `out` states).
2. **Support Functions:** Implement the mathematical support functions (such as *Gen*, *Kill*, intersections, and unions) for the specific analyses.
3. **Annotate CFG:** Implement a function that takes a plain CFG and returns a CFG fully annotated with the analysis results.
4. **Required Analyses:** You must implement the following three analyses:
   - Defined Variables
   - Live Variables
   - Reaching Definitions

Report Documentation

In the final project report, you must explicitly detail your implementation choices for this fragment: * **Annotations:** How did you manage the block annotations programmatically? * **Edge Cases:** What edge cases did you encounter and handle while formalizing the support functions (e.g., resolving issues with maximal blocks)? * **Representation:** What data representation did you choose for Reaching Definitions (e.g., unique assignment identifiers) and why?

Fragment 7: Optimizations

Prerequisite: Fragment 6 (Data-Flow Analysis).

This fragment focuses on leveraging the results of the data-flow analyses computed in the previous step to perform error checking and apply code transformations. The goal is to improve the quality of the MiniImp code (e.g., performance, memory footprint) before the final translation to machine code.

Implementation Requirements

You must implement an optimization pipeline that includes the following specific steps:

1. **Undefined Variables Check:** Implement a function that, given an annotated CFG, checks whether there are possibly undefined variables being used and handles/reports them.
2. **Dead Store Elimination:** Implement a compiler step that removes useless variable assignments (stores) whose values are never read later.
3. **Constant Folding:** Implement a compiler step that evaluates constant expressions at compile time (e.g., replacing `2 + 3` with `5`).
4. **Constant Propagation:** Implement a compiler step that replaces the uses of a variable with a constant value if that variable is known to hold a constant at that specific point in the program.
5. **Optimization Pipeline:** Build an infrastructure to orchestrate these passes. Since optimizations can cascade (e.g., constant propagation might open new opportunities for dead store elimination), you need a system to control their execution.

Report Documentation

In the final project report, you must explicitly detail your implementation choices for the optimization phase:

- **Undefined Variables:** Which specific data-flow analysis is used to detect undefined variables? How does your compiler handle them when found?
- **Compiler Infrastructure:** What is the underlying architecture you built to support optimizations?
- **Pass Control:** How do you control the sequence of optimization passes?
- **Strategy:** If you defined a specific optimization strategy (e.g., executing passes in a specific loop or order until a fixpoint is reached), what is it and why did you choose it?
- Any other relevant implementation details.

Fragment 8: LLVM IR Code Generation

Prerequisite: Fragment 6 (Data-Flow Analysis).

The final milestone of the MiniImp project is the translation of the abstract syntax (or the annotated CFG) into valid, executable LLVM Intermediate Representation.

Implementation Requirements

1. **IR Emitter:** Write a function that traverses the program and outputs LLVM IR text (`.ll` format).
2. **Expression Flattening:** LLVM instructions are RISC-like. Complex expressions must be broken down into simple operations using fresh virtual registers (`%1`, `%2`, etc.).
3. **Control Flow:** Translate `if-then-else` and `while` constructs into labeled basic blocks with explicit conditional and unconditional branches.
4. **Variable Management (The `alloca` trick):** To maintain the strict Static Single Assignment (SSA) form without manually computing ϕ-nodes:
   - Allocate all local variables on the stack at the entry block using `alloca`.
   - Use explicit `load` and `store` instructions for all reads and assignments.

5. **Optimization Integration:** Apply at least the `mem2reg` optimization via the `opt` tool. This automatically promotes the stack memory operations into pure SSA registers.

Testing and Linking (C Wrapper)

Because MiniImp does not have built-in I/O functions, you must interface with external C code to verify execution:

```c
// wrapper.c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

// This matches the LLVM function generated by your compiler
extern int64_t func(int64_t);

int main(int argc, char *argv[]) {
    int64_t inp, out;
    scanf("%ld", &inp);
    out = func(inp);
    printf("%ld\\n", out);
    return EXIT_SUCCESS;
}
```

Compilation Pipeline:

1. `opt -passes=mem2reg program.ll -S -o optimized.ll`
2. `llc -filetype=obj optimized.ll -o miniimp.o`
3. `clang wrapper.c miniimp.o -o program`

Report Documentation

In the final project report, detail your code generation strategy:

- How did you handle the generation of fresh names?
- What was the impact of your block size?
- What technique did you choose to produce SSA-form IR?
- Any other details you consider important.